

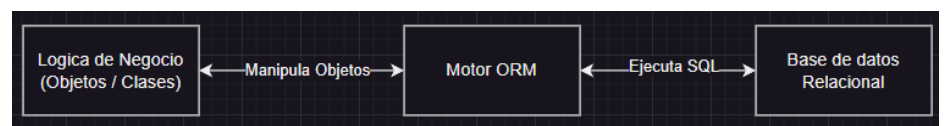
b) Investigación:

- Patrón DAO:
- ¿Qué es un Patrón en desarrollo de software?
 - **Un patrón de diseño es una solución general, reutilizable y estructurada para un problema común que ocurre dentro de un contexto dado. No es código terminado, sino una plantilla o descripción lógica de cómo resolver un problema arquitectónico o de diseño**
- ¿Qué es el patrón de acceso a datos DAO? ¿Para qué sirve? Realizar de forma gráfica su funcionamiento.
 - **DAO (Data Access Object) es un patrón arquitectónico estructural que aísla la capa de lógica de negocio o servicios de la capa de persistencia (la base de datos).**
 - **Sirve para abstraer y encapsular todos los accesos a la fuente de datos. A través de una interfaz, la aplicación interactúa con la base de datos sin conocer los detalles de implementación (como sentencias SQL o configuraciones de conexión). Esto permite que la base de datos pueda ser modificada o reemplazada sin afectar la lógica principal del sistema, mejorando el mantenimiento y facilitando las pruebas unitarias (testing)**



- Object Relational Mapping (ORM):
 - ¿Qué es un ORM? ¿Cómo funciona un ORM?
 - **ORM (Mapeo Objeto-Relacional) es una técnica de programación que permite aislar al programador del manejo de bases de datos relacionales y su lenguaje nativo. Funciona como una capa intermedia que convierte de forma transparente los objetos de un lenguaje de programación orientado a objetos en tablas y registros de una base de datos. En lugar de escribir sentencias SQL manuales, el desarrollador interactúa con objetos (instancias de clases), y el ORM se encarga de traducir esas interacciones a consultas SQL válidas para persistir, actualizar o recuperar los datos**

- ¿Cuáles son los ORM más conocidos del mercado? ¿Qué lenguajes de programación pueden interactuar con ORM?
 - **ORM más conocidos y lenguajes soportados**
 - **Java:** Hibernate, EclipseLink
 - **Python:** SQLAlchemy, Peewee
 - **C#:** Entity Framework
 - **PHP:** Eloquent
 - **JavaScript/TypeScript:** TypeORM, Sequelize
 - **Ruby:** Active Record
 - **Go:** Gorm
- ¿Cuáles son sus ventajas y desventajas? Realizar de forma gráfica su funcionamiento.
 - **Ventajas:**
 - **Productividad:** Reduce enormemente la cantidad de código repetitivo (boilerplate) de acceso a datos
 - **Independencia de la Base de Datos:** Facilita el cambio de motor de base de datos (ej. de MySQL a PostgreSQL) cambiando solo la configuración
 - **Seguridad:** Previene inyecciones SQL nativamente
 - **Paradigma unificado:** Permite al desarrollador pensar exclusivamente en objetos
 - **Desventajas:**
 - **Rendimiento:** Puede generar consultas SQL ineficientes o el problema de las “consultas N+1” si no se configura correctamente
 - **Curva de aprendizaje:** Entender a fondo cómo el ORM maneja la caché y las sesiones es complejo
 - **Pérdida de control:** Para consultas extremadamente complejas o analíticas, el lenguaje SQL nativo suele ser mejor

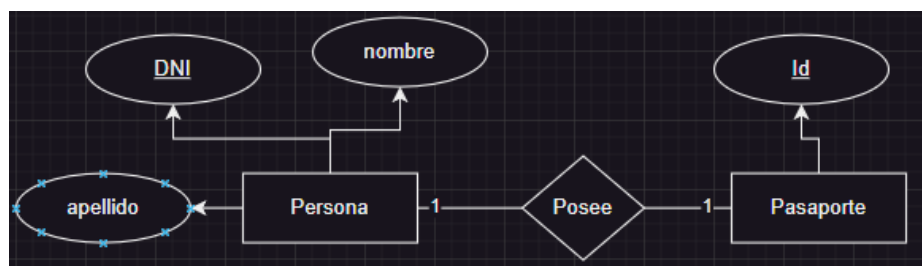


- **Patrones Utilizados en ORM:**
 - ¿Cuáles son los patrones principales utilizados para que los ORM permitan a los programadores trabajar con objetos, realizando la abstracción para no utilizar las sentencias del lenguaje SQL? Realizar un cuadro explicando cada uno de ellos.

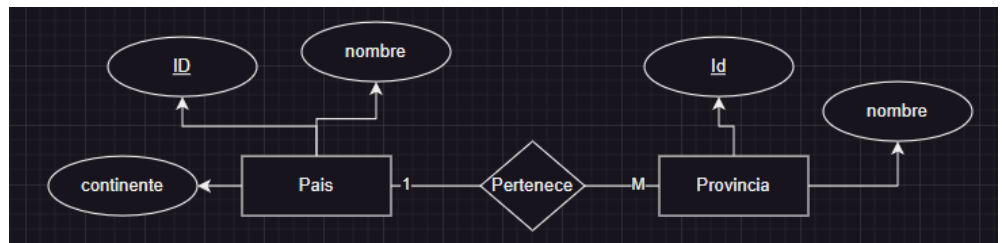
Patrón	Descripción / Funcionamiento
Representación de Objetos como Tablas	Propone la definición de una tabla en la base de datos relacional por cada clase de objeto persistente. Los atributos del objeto que contienen datos primitivos (número, cadena, booleano) se mapean directamente como columnas en la tabla correspondiente
Identificador de Objeto (OID)	Asegura la correspondencia única asignando un identificador de objeto (OID) a cada registro. Normalmente es un valor alfanumérico o numérico. Cada tabla tendrá este OID como clave primaria, garantizando que no se materialicen objetos duplicados en memoria
Representación de Relaciones en Tablas	Mapea las asociaciones orientadas a objetos en relaciones relacionales. Para relaciones 1:1 y N:1, utiliza claves foráneas (Foreign Keys). Para colecciones (1:N) o relaciones N:M, crea tablas asociativas intermedias que registran los OIDs de los objetos vinculados

- Java Persistence Api (JPA):
 - ¿Qué es JPA? ¿Cuáles son las principales características de JPA?
 - **JPA es un estándar de la industria que define interfaces genéricas y anotaciones para el manejo de la persistencia y el mapeo objeto-relacional en Java. Define "qué se debe hacer" de forma estándar, mientras que los proveedores concretos (como Hibernate) implementan "cómo se hace"**
 - **Sus características principales incluyen:**
 - **Mapeo basado en anotaciones (metadata) sobre clases planas**
 - **Una API unificada a través del EntityManager**
 - **Un lenguaje robusto de consultas orientado a objetos (JPQL)**
 - **Soporte estandarizado para transacciones y control de concurrencia**
 - ¿Qué significa que una clase sea de tipo Entidad? ¿Cuáles son las anotaciones que se pueden utilizar en una clase de tipo entidad? ¿Cuáles son las anotaciones que se pueden utilizar para validar información en un atributo de JPA?

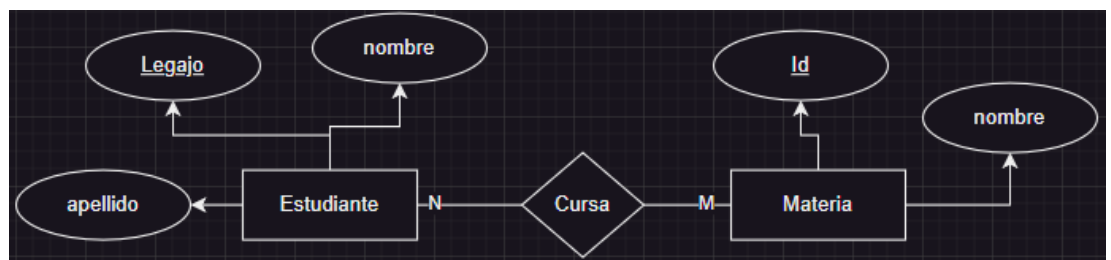
- Una clase de tipo Entidad es un objeto de dominio ligero (un simple POJO de Java) que representa de forma directa un registro dentro de una tabla en la base de datos. Su estado es manejado por el contexto de persistencia de JPA
- Anotaciones de Mapeo Estructural:
 - @Entity: Marca la clase como persistente
 - @Table: Permite especificar el nombre físico de la tabla
 - @Id: Marca un atributo como la clave primaria (OID)
 - @Column: Define propiedades físicas de la columna (nombre, longitud)
 - @Temporal: Especifica cómo guardar fechas (DATE, TIME, TIMESTAMP)
- Anotaciones para Validación de Información
 - En JPA, las validaciones pueden abordarse en dos niveles. A nivel de esquema de base de datos usando parámetros en @Column (ej. @Column(nullable = false, length = 100, unique = true)). A nivel de ejecución de objetos, JPA se integra de forma nativa con el estándar "Jakarta Bean Validation", permitiendo usar anotaciones como:
 - @NotNull (no permite valores nulos)
 - @Size(min=, max=) (restringe tamaño de cadenas)
 - @Pattern(regex=...) (valida mediante expresiones regulares)
 - @Min y @Max (para valores numéricos)
- ¿Cuáles son las relaciones entre clase que se pueden anotar en JPA para ser mapeada a una base de datos? ¿Dar un ejemplo sencillo de cada una de las relaciones y realizar el DER para cada ejemplo?
 - 1. Uno a Uno (@OneToOne): Relaciona una entidad de forma exclusiva con otra.
 - Ejemplo: Una "Persona" tiene un único "Pasaporte".



- 2. Uno a Muchos (@OneToMany) / Muchos a Uno (@ManyToOne): El lado 1 tiene una colección, el lado N tiene la referencia simple.
 - Ejemplo: Un "País" posee muchas "Provincias". Cada "Provincia" pertenece a un "País".

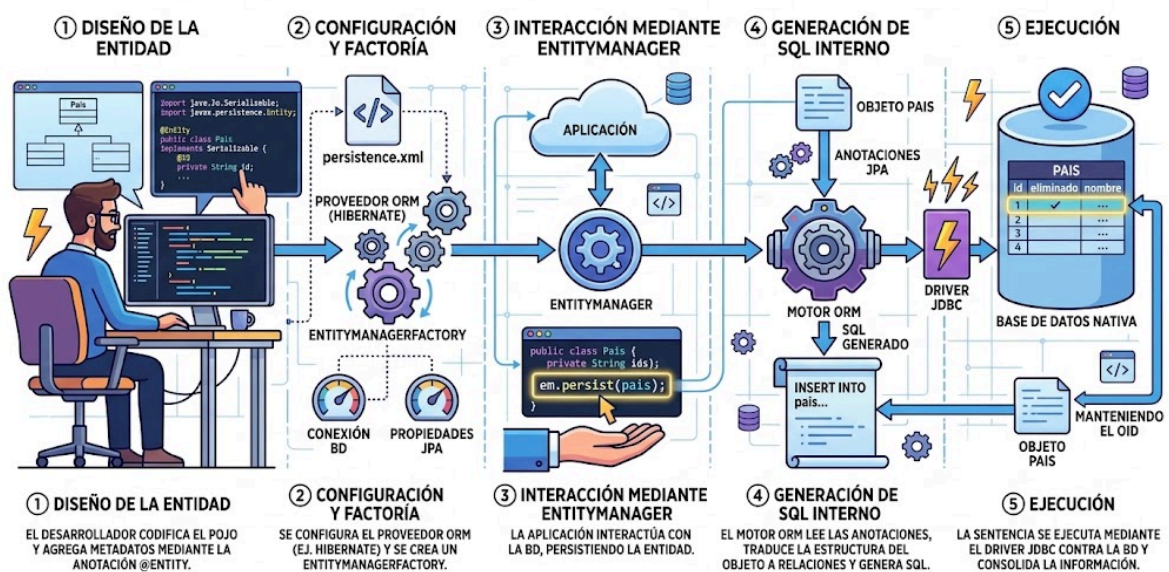


- 3. Muchos a Muchos (@ManyToMany): Ambas entidades tienen colecciones. Se requiere una tabla intermedia.
 - Ejemplo: Un "Estudiante" puede cursar varias "Materias" y una "Materia" puede ser cursada por varios "Estudiantes".



- ¿Graficar paso por paso cómo es el funcionamiento de JPA para el Mapeo de una clase Entidad hasta su conversión en tabla y sus respectivas relaciones?

FLUJO DE MAPEO JPA (PASO A PASO)



- ¿Qué es un JPAQL? ¿Qué similitudes y que diferencias existen con una sentencia SQL?. Dar 5 ejemplos de JPAQL y su respectiva sentencia SQL.
 - **JPQL (Java Persistence Query Language)** es el lenguaje de consultas orientado a objetos definido por la especificación JPA. Funciona a través de los métodos del EntityManager, como `createQuery()`
 - **Similitudes con SQL:** Comparte palabras reservadas y estructura fundamental (SELECT, FROM, WHERE, GROUP BY, ORDER BY, JOIN)
 - **Diferencias principales:** A diferencia de SQL que consulta Tablas y Columnas físicas, JPQL consulta Entidades (Clases Java) y sus atributos directamente. JPQL es agnóstico del motor de base de datos; el ORM se encarga de compilar el JPQL en el SQL dialectal correcto para Oracle, MySQL, Postgres, etc

Operación	Sentencia JPQL (Orientado a Clases)	Sentencia SQL (Orientado a Tablas)
Seleccionar Todos	SELECT p FROM Pais p	SELECT * FROM tabla_pais
Filtrar con Condición (WHERE)	SELECT p FROM Pais p WHERE p.eliminado = FALSE	SELECT * FROM tabla_pais WHERE eliminado = 0
Búsqueda Parametrizada	SELECT e FROM Empleado e WHERE e.nombre = :nom	SELECT * FROM tabla_empleado WHERE nombre = ?
Funciones de Agregación	SELECT COUNT(p) FROM Provincia p	SELECT COUNT(id) FROM tabla_provincia
Actualización (UPDATE)	UPDATE Propietario p SET p.telefono = :tel WHERE p.id = :id	UPDATE tabla_propietario SET telefono = ? WHERE id = ?